

А. Биби и его папа

2 секунды, 256 мегабайт

Биби вырос, наступила пора покинуть родительское гнездо и отправиться навстречу новым знаниям. Пин не хочет потерять с сыном связь, поэтому решил смастерить для него телеграф.

Чтобы научиться пользоваться телеграфом, Биби должен последовательно отправить с его помощью n сообщений $s_1, s_2, \dots, s_n$.

Пин предусмотрел, что с большой вероятностью Биби захочет отправлять одинаковые сообщения, например «hello father», несколько раз, поэтому добавил кнопку «↑», которая загружает в телеграф предыдущее отправленное сообщение.

Таким образом, Биби может отправить сообщение s одним из двух способов:

- набрать сообщение посимвольно и нажать «Enter», выполнив суммарно $|s| + 1$ ($|s|$ — это длина строки s) нажатие;
- если Биби уже отправлял такое же сообщение ранее, нажать на «↑» произвольное количество раз k , загрузив в телеграф k -е по счёту с конца отправленное сообщение, и нажать «Enter», выполнив суммарно $k + 1$ нажатие.

Помогите Биби последовательно отправить все сообщения, выполнив как можно меньшее суммарное число нажатий.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных.

В первой строке каждого набора дано целое число n ($1 \leq n \leq 2 \cdot 10^5$) — количество сообщений.

В следующих n строках каждого набора дана последовательность сообщений $s_1, s_2, \dots, s_n$, состоящих из строчных и заглавных букв латинского алфавита, а также пробелов. Сообщения разделены переводами строк.

Гарантируется, что сумма длин сообщений по всем наборам входных данных не превосходит $2 \cdot 10^5$.

Также гарантируется, что первый и последний символы каждого сообщения не являются пробелами.

Выходные данные

Для каждого набора входных данных выведите единственное целое число — минимальное количество нажатий, необходимое для отправки всех сообщений.

входные данные

```
4
6
hi
hi
hello father
i am fine
hello father
hi
3
aba
b
aba
6
krosh
pin
ezik
krosh and ezik
pin
goodbye
4
hello krosh
goodbye
hello ezik
goodbye
```

выходные данные

```
34
9
42
34
```

Во втором наборе входных данных оптимальна следующая последовательность действий: «a», «b», «a», «Enter», «b», «Enter», «↑», «↑», «Enter».

В третьем наборе входных данных одна из оптимальных последовательностей: «k», «f», «o», «s», «h», «Enter», «p», «i», «n», «Enter», «e», «z», «i», «k», «Enter», «k», «f», «o», «s», «h», «↑», «a», «n», «d», «↑», «e», «z», «i», «k», «Enter», «↑», «↑», «↑», «Enter», «g», «o», «o», «d», «b», «y», «e», «Enter».

Обратите внимание, что сообщения могут содержать пробелы. Для считывания набора входных данных на языке C++ вы можете использовать представленный ниже сниппет.

```
#include <iostream>
#include <string>
using namespace std;

void solve() {
    int n; cin >> n;
    string tmp; getline(cin, tmp);
    for (int i = 0; i < n; ++i) {
        string s; getline(cin, s);
        // ...
    }
}
```

В. Забытая история

2 секунды, 256 мегабайт

Карыч забыл что-то очень важное и сильно обеспокоился из-за этого. Смешарики решили ему помочь.

Они подключили Карыча к компьютеру Лосяша и обнаружили n воспоминаний $s_1, s_2, \dots, s_n$, представленных в виде строк.

Карыч может переместиться из воспоминания s_i в воспоминание s_j , если существует такой символ c , что $f(s_i, c) > 0$ и $f(s_j, c) = f(s_i, c)$, где $f(s, c)$ — количество вхождений символа c в строку s . После такого перемещения компьютер Лосяша нагреется на $f(s_i, c)$ градусов. Если возможных перемещений несколько, Карыч может выбрать любое.

Расстоянием от воспоминания s_i до воспоминания s_j называется суммарный нагрев компьютера Лосяша на пути Карыча от s_i к s_j .

Карыч забыл, какое из воспоминаний является важным. Помогите ему определить **минимальное** расстояние от первого воспоминания до всех остальных. Если до каких-то воспоминаний добраться невозможно, определите это.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных.

В первой строке каждого набора дано целое число n ($1 \leq n \leq 10^5$) — количество воспоминаний.

В следующих n строках даны воспоминания $s_1, s_2, \dots, s_n$, состоящие из строчных букв латинского алфавита.

Гарантируется, что сумма длин s_i по всем наборам входных данных не превосходит 10^6 .

Выходные данные

Для каждого набора входных данных выведите $n - 1$ целых чисел – минимальные расстояния до воспоминаний $s_2, s_3, \dots, s_n$ соответственно. Если какое-то воспоминание недостижимо, выведите вместо минимального расстояния «-1».

Входные данные
3
2
ulyanovskisthebest
moscowisthebest
2
abcd
efgh
5
ull
llyyy
yyyaaaa
aaaaannnnn
nnnnnnovsk
Выходные данные
1
-1
2 5 9 14

С. Горы и конфеты

1 секунда, 256 мегабайт

Нюша поссори́лась с Барашем, объе́лась конфет и заскучала. У неё осталось бесконечное количество конфет каждого цвета. Цвет конфеты определяется целым числом от 1 до n .

Чтобы развлечься, она выложила из конфет квадрат $n \times n$, оставив какие-то клетки пустыми. Каждый цвет встречается в квадрате n раз или не встречается вообще.

Квадрат называется *красивым*, если цвета в каждой строке и каждом столбце образуют перестановку чисел от 1 до n .

Помогите Нюше дополнить квадрат до *красивого* или сообщите, что это невозможно.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных.

В первой строке каждого набора дано целое число n ($1 \leq n \leq 100$) — размер квадрата.

В следующих n строках каждого набора даны n целых чисел $a_{i,1}, a_{i,2}, \dots, a_{i,n}$ ($0 \leq a_{i,j} \leq n$) — цвета конфет в i -й строке квадрата. Если $a_{i,j} = 0$, то конфета в соответствующей ячейке отсутствует.

Гарантируется, что каждый присутствующий в квадрате цвет встречается ровно n раз.

Также гарантируется, что в каждой строке и в каждом столбце нет повторяющихся цветов.

Также гарантируется, что сумма n^2 по всем наборам входных данных не превосходит 10^4 .

Выходные данные

Для каждого набора входных данных выведите в первой строке «YES», если Нюша сможет дополнить квадрат до *красивого*, и «NO» иначе.

При положительном ответе выведите n строк, содержащих по n чисел — вариант заполнения квадрата. Если ответов несколько, вы можете вывести любой.

Входные данные				
3				
3				
1	2	3		
2	3	1		
3	1	2		
1				
0				
5				
0	4	2	0	0
0	2	0	4	0
4	0	0	2	0
2	0	0	0	4
0	0	4	0	2
Выходные данные				
YES				
1	2	3		
2	3	1		
3	1	2		
YES				
1				
YES				
5	4	2	3	1
3	2	1	4	5
4	1	5	2	3
2	5	3	1	4
1	3	4	5	2

D. Утеранные извинения

2 секунды, 256 мегабайт

Это особая задача. Внимательно прочитайте примечание (замечание).

Совунья случайно нашла в старых вещах n писем от своего ухажёра из прошлого. Её накрыли воспоминания о n памятных датах, и в порыве чувства упущенного счастья она решила уничтожить все письма как можно скорее.

Каждое письмо соединяет ребром какие-то памятные даты v и u .
Каждую пару памятных дат соединяет не более одного письма.

Памятные даты v и u находятся в одной компоненте связности, если можно добраться из v в u по рёбрам. Известно, что изначально **все** памятные даты находятся в одной компоненте связности.

За одно действие Совунья может порвать **не более** половины писем в **каждой** компоненте связности. Половина округляется вверх до ближайшего целого.

Помогите Совунье порвать **все** письма за как можно меньшее количество действий.

Входные данные

В первой строке дано целое число n ($3 \leq n \leq 2 \cdot 10^5$) — количество памятных дат и писем.

В следующих n строках даны целые числа v, u ($1 \leq v, u \leq n, v \neq u$) — описание писем.

Гарантируется, что **все** памятные даты находятся в одной компоненте связности.

Выходные данные

В первой строке выведите целое число k ($1 \leq k \leq 20$) — количество действий.

Далее выведите описание действий.

В первой строке i -й итерации выведите целое число c_i ($1 \leq c_i \leq n$) — количество удалённых писем.

Во второй строке i -й итерации выведите c_i целых чисел $d_1, d_2, \dots, d_{c_i}$ ($1 \leq d_i \leq n$) — номера удалённых писем в порядке ввода.

Входные данные	
5	
1 2	
3 4	
5 1	
2 3	
3 5	

выходные данные
4
2
1 4
1
2
1
5
1
3

В этой задаче всего один скрытый тест, сгенерированный случайно. Вы будете соревноваться на нём с другими участниками олимпиады. В разделе «Протокол» для каждой успешной посылки вы увидите количество операций, за которое справились. Для посылок со статусом «Неправильный ответ» вы увидите короткое сообщение чекера, которое может помочь при дебаге.

Во время тура ваше решение проверяется на корректность, но не на минимальность. Вы можете получить вердикт «Ожидает подтверждения». Он означает, что решение вывело корректную последовательность операций. Если вы сдадите корректное решение повторно, статус предыдущего решения поменяется на «Проигнорировано». К финальному тестированию на минимальность будет допущено ваше последнее **корректное** решение.

Финальное тестирование будет проведено после окончания тура. Участники, порвавшие все письма за минимальное количество действий среди всех участников, получают вердикт «ОК». Остальные участники, сдавшие корректное решение, получают вердикт «Неправильный ответ». Решение жюри **не участвует** в соревновании. Например, если корректное решение сдал всего один участник, он гарантированно получит вердикт «ОК», но если корректное решение сдали четыре участника, из них первые два справились за шесть действий, третий за семь, а четвёртый за двадцать, вердикт «ОК» получают только первые два.

Штраф считается по обычным правилам, как-будто посылка с вердиктом «Ожидает подтверждения» является посылкой с вердиктом «ОК», при этом посылки с вердиктом «Проигнорирована» игнорируются.

Е. Игра

2 секунды, 256 мегабайт

Это интерактивная задача.

Крош пришёл в очередной раз поиграть в компьютер Лосяша, но вместо привычных игр «Принц для Юши 2» или «Крутой спуск» он обнаружил новую игру «Коварные демонюги». Крошу очень понравилась эта игра, но он постоянно проигрывал. Помогите Крошу составить стратегию, с которой он будет выигрывать чаще, чем проигрывать.

В данной задаче вам будут предоставлены правила игры против интерактора в «Коварные демонюги». Чтобы решить эту задачу, вам нужно выиграть у интерактора хотя бы 500 из 1000 игр. Интерактор **не является** адаптивным, то есть не подстраивается под ваш стиль игры.

«Коварные демонюги» — карточная игра, в которой карты пронумерованы и выложены по кругу. В этой игре у каждой карты есть своя роль.

Представим роли:

- Рыцарь**. Это персонаж, которого нельзя *убить*, если он не *врёт*.
- Охотник**. Это персонаж, который скажет, на каком расстоянии от него находится **Демон**, но не скажет, в каком направлении. Если этот персонаж врёт, он скажет случайное число от 1 до $\frac{n}{2}$ с округлением вниз.
- Гренадер**. Если вы убьёте этого персонажа, когда он не врёт, игра заканчивается поражением.
- Шута**. Персонаж, который позволяет выбрать три различные карты, и он скажет, сколько **демонов** среди них. Способность

Шута можно использовать только один раз за игру. Если будут выбраны не различные карты, игра закончится поражением.

- Судья**. Персонаж, который позволяет выбрать одну карту и узнать, *врёт* ли она. Эту способность можно использовать только один раз за игру.
- Демон**. Эта карта всегда *врёт* и будет притворяться другой ролью, а также заставляет соседние слева и справа карты *врать*. Когда карта *врёт*, **Судья**, **Охотник** и **Шут** предоставляют неправильную информацию; **Рыцаря** можно будет убить; **Гренадер** не закончит игру вашим поражением.

Изначально карты появляются в закрытом виде, а у вас есть 9 жизней. Кроме того, в игре только один **Демон**. За один ход игрок может выполнить одно из трёх действий:

- Открыть неоткрытую карту. При этом игрок теряет 2 жизни.
- Убить любую карту (открытую или закрытую):
 - Если игрок пытается убить **Демона**, то игрок побеждает.
 - Если игрок пытается убить **Рыцаря**, который врёт, то игрок теряет 5 жизней.
 - Если игрок пытается убить **Рыцаря**, который не врёт, то игрок не теряет жизней.
 - Если игрок пытается убить **Гренадера**, который не врёт, то игрок проигрывает.
 - Иначе игрок теряет 3 жизни.
- Использовать способность открытой карты. За это действие жизни не тратятся.

Если после вашего хода у вас меньше одной жизни, вы проиграли. Иначе вы можете делать ходы дальше.

Входные данные

В первой строке каждой игры вводится единственное число n ($5 \leq n \leq 8$) — количество карт в игре.

В этой задаче четыре теста с разными n .

Тест	Ограничения
1	$n = 5$
2	$n = 6$
3	$n = 7$
4	$n = 8$

Протокол взаимодействия

Интерактор **не адаптивен**, то есть не пытается подстроить состояние игры против вас. Состояние игры предreshено в начале игры и не меняется в зависимости от ваших действий.

Если в какой-то момент вы получили на вход слово «stop», это значит, что вы проиграли, и необходимо завершить программу. Например, у вас закончилось здоровье или вы сделали некорректный ход.

Вы можете использовать три команды:

- `kill x` — вы убиваете карту под номером x (считая с единицы).
- `unc x` — вы переворачиваете карту под номером x , если она не была перевёрнута, иначе ваш ход заканчивается. После этого действия вы получаете 2 урона.
- `use pos` — вы используете способность **Судьи** или **Шута** на позиции `pos`, после этого вам предложат ввести одно или три числа для соответствующих ролей. Обратите внимание: если нужной роли на позиции `pos` не существует или она ещё не открыта, то игра сразу закончится.

Если была использована команда `kill`, вы получите сообщение «win», если вы убили Демона и нужно перейти к следующей игре, или «miss x » ($x < 0$), если вы этого не сделали: вы получаете $|x|$ урона.

Если была использована команда `unc`, вы получите информацию о выбранной карте:

- 'К' — **Рыцарь**;
- 'Н' — **Охотник**;
- 'S' — **Гренадер**;
- 'В' — **Шут**;
- 'J' — **Судья**.

Обратите внимание, что ни одна карта не вскрыется сама как **Демон**.

Если вы получили на вход букву 'Н', следом вы получите число x — расстояние, на котором находится **Демон**.

Если была использована команда `use`, вам нужно ввести одно или три числа для **Судьи** и **Шута** соответственно:

- Если была использована способность **Шута**, вернётся одно число — количество **Демонов** среди 3 выбранных карт, число от 0 до 3.
- Если была использована способность **Судьи**, вернётся одно число — 0, если карта врёт, 1, если карта не врёт.

Не забывайте сбрасывать буфер после каждого вывода. Для этого можете использовать `std::endl` в C++ или стандартную функцию `print` в Python. Не используйте `ios_base::sync_with_stdio(false)` в C++.

входные данные
5
miss -3
Н 2
win
...
выходные данные
kill 2
unc 4
kill 1
...

Что произошло в примере:

Первым действием мы убили карту на позиции 2, это был не **Демон** и не **Рыцарь**.

Вторым действием мы открыли карту на позиции 4, и это оказался **Охотник**, который сказал, что на расстоянии 2 находится **Демон**.

Третьим действием мы убили **Демона**; оказалось, что **Охотник** не соврал.

Г. Коллекция

2 секунды, 256 мегабайт

Ёжик долго собирал свою коллекцию. В ней представлены n фантиков, которые Крош оценил в $a_1, a_2, \dots, a_n$ морковок.

Крош и Ёжик решили доказать свою дружбу, разделив **все** фантики из коллекции между собой *честно*. Разделение называется *честным*, если модуль разности суммарной стоимости фантиков Кроша и фантиков Ёжика не превосходит 100.

Помогите друзьям разделить фантики *честно*.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных.

Во второй строке каждого набора даны n целых чисел $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 100$) — стоимости фантиков в морковках.

Гарантируется, что сумма n по всем наборам входных данных не превосходит $2 \cdot 10^5$.

Выходные данные

Для каждого набора входных данных выведите в первой строке целое число — модуль разности для *честного* разбиения.

В следующей строке выведите n символов 'К' или 'Е', обозначающих, что Крош или Ёжик взяли i -й фантик.

Можно показать, что при указанных ограничениях *честное* разделение всегда существует.

Если ответов несколько, вы можете вывести любой. Обратите внимание, что вам **не требуется** минимизировать модуль разности.

входные данные
3
1
1
2
100 100
5
6 27 61 14 15
выходные данные
1
Е
0
КЕ
11
ЕКЕКК

В третьем наборе входных данных Ёжик взял фантики стоимостью $6 + 61 = 67$ морковок, а Крош — стоимостью $27 + 14 + 15 = 56$ морковок. Модуль разности стоимостей $67 - 56 = 11$, $11 \leq 100$, значит, разбиение является *честным*.

Г. Железная няня

1 секунда, 256 мегабайт

Страшные дела творятся в Ромашковой долине.

Железная няня активируется в точке $(0, 0)$ и начинает искать бодрствующих смешариков. Смешарики прячутся в каких-то точках и не меняют свою позицию.

Далее происходят q событий, события бывают двух видов:

1. Железная няня обнаруживает бодрствующего смешарика в точке (x, y) , при этом остаётся на месте;
2. Железная няня перемещается в *ближайшую* точку с бодрствующим смешариком и укладывает его спать; если таких точек несколько, Железная няня перемещается в лексикографически минимальную.

Ближайшей к точке (x_1, y_1) называется точка (x_2, y_2) , которая минимизирует величину $|x_1 - x_2| + |y_1 - y_2|$ (Манхэттенское расстояние).

Определите точки, в которых смешарики будут уложены спать.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных.

В первой строке каждого набора дано целое число q ($1 \leq q \leq 10^5$) — количество событий.

В следующих q строках каждого набора даны события:

- $1\ x\ y$ ($1 \leq x, y \leq 10^5$) — Железная няня обнаруживает нового бодрствующего смешарика;
- 2 — Железная няня укладывает спать *ближайшего* бодрствующего смешарика.

Гарантируется, что сумма q по всем наборам входных данных не превосходит 10^5 .

Также гарантируется, что в одном наборе входных данных в событиях первого типа все точки (x, y) **попарно различны**.

Также гарантируется, что перед любым событием второго типа известно положение хотя бы одного бодрствующего смешарика.

Выходные данные

Для каждого события второго типа выведите точку, в которую переместилась Железная няня.

входные данные	
1	
9	
1 1 2	
1 2 1	
2	
1 4 1	
1 3 2	
2	
1 2 4	
2	
2	
выходные данные	
1 2	
2 1	
3 2	
4 1	

Изначально няня находится в точке $(0, 0)$.

Перед первым удалением бодрствующие смешарики находятся в точках $(1, 2)$ и $(2, 1)$. Они равноудалены от няни, поэтому выбирается лексикографически минимальная точка $(1, 2)$.

Перед вторым удалением бодрствующие смешарики находятся в точках $(2, 1)$, $(3, 2)$ и $(4, 1)$. Ближайшие точки — $(2, 1)$ и $(3, 2)$, поэтому выбирается $(2, 1)$.

Перед третьим удалением бодрствующие смешарики находятся в точках $(2, 4)$, $(3, 2)$ и $(4, 1)$. Выбирается $(3, 2)$.

Перед четвёртым удалением бодрствующие смешарики находятся в точках $(2, 4)$, $(4, 1)$. Выбирается $(4, 1)$.

Н. Большой куш

1 секунда, 256 мегабайт

Это задача с открытыми тестами.

Пин построил игровой аппарат, в котором можно выиграть морковку. Для обеспечения честности он зафиксировал несколько псевдослучайных булевых функций, определяющих результат игры. Функция под номером t задаётся от n_t булевых переменных. Игра должна быть непредсказуемой и честной, поэтому функций не должен знать никто, кроме Пина и вас.

Чтобы скрыть реализацию от игроков, Пин при постройке автомата использует только nand-компоненты. Каждый такой компонент принимает два входных бита и возвращает один бит в соответствии с таблицей ниже.

a	b	a nand b
0	0	1
0	1	1
1	0	1
1	1	0

Две булевы функции f_1 и f_2 называются *эквивалентными*, если

$$f_1(x_1, \dots, x_{n_1}) = f_2(x_1, \dots, x_{n_1})$$

для любых значений переменных $x_1, x_2, \dots, x_{n_1} \in \{0, 1\}$.

Для каждого теста по заданному номеру t постройте схему из nand-компонент, которая будет задавать функцию, *эквивалентную* указанной в таблице. Количество использованных nand-компонент не должно превышать заданный лимит, потому что вместимость гаража Пина конечна.

Номер	n_t	Функция	Лимит nand-компон
0	3	$(x_1 x_2) \& x_3$	20
1	2	$(x_1 x_2)$	3
2	4	$x_4 \oplus ((x_1 x_2 x_4) \& x_3)$	15
3	5	$x_1 \oplus x_2 \oplus x_4 \oplus (x_1 \& x_3) \oplus (x_2 \& x_5) \oplus (x_3 \& x_4 \& x_5) \oplus 1$	25
4	6	$((x_1 x_2 x_3) \& (x_2 x_3 x_5 x_6)) \oplus (x_1 \& x_4)$	21
5	6	$(\neg(x_1 \& x_2) x_3) \oplus ((x_4 \& x_5 \& x_6) (x_4 \oplus x_6))$	14
6	8	$(x_1 \oplus x_3) ((x_2 \& x_5) \oplus x_6) ((x_3 x_4 x_5 x_6 x_7 x_8) \oplus (x_2 \& x_6))$	33

Входные данные

В единственной строке дано целое число t ($0 \leq t \leq 6$) — номер функции, для которой необходимо построить схему.

Выходные данные

Для функции под номером t выведите в первой строке целое число m_t — количество используемых nand-компонент. Оно не должно превышать соответствующий лимит для функции.

В следующих m_t строках выведите описание nand-компонент в формате « x_a nand x_b ». Входные переменные функции под номером t заданы переменными $x_1, x_2, \dots, x_{n_t}$ и могут быть использованы в качестве входных битов для nand-компонент.

Все nand-компоненты исполняются последовательно. Результат выполнения каждого компонента записывается в переменную с **минимальным** неиспользованным номером. Результатом всех вычислений является результат выполнения последнего компонента.

входные данные	
0	
выходные данные	
5	
x_1 nand x_1	
x_2 nand x_2	
x_4 nand x_5	
x_6 nand x_3	
x_7 nand x_7	

Ниже представлены таблицы истинности для битовых операций, используемых в условии.

a	b	a nand b	a & b	a b	¬ a	a ⊕ b
0	0	1	0	0	1	0
0	1	1	0	1	1	1
1	0	1	0	1	0	1
1	1	0	1	1	0	0

I. Право на одиночество

3 секунды, 256 мегабайт

Бараш наконец-то смог уединиться и тут же взялся за сочинение долгожданного стихотворения. Его новый стих содержит 2^n строк, в которых $a_1, a_2, a_3, \dots, a_{2^n}$ слогов соответственно.

Поэты привыкли к одностишьям, двустишьям, четверостишьям и т.д., поэтому прекрасно работают со степенями двойки. Бараш построил над своим стихом полное бинарное дерево, записав в листья значения $a_1, a_2, \dots, a_{2^n}$. Напомним, что полным бинарным деревом называется бинарное дерево, каждая вершина которого имеет 0 или 2 сына.

Согласно новой литературной теории Бараша, стих *шедевральный*, если все a_i в полном бинарном дереве попарно различны. Эта гениальная идея пришла к нему довольно поздно, поэтому было проще дописать стих и потом немного подправить его, чем переписывать с нуля.

Бараш выбрал для себя действие, которым будет редактировать стих. Он рассматривает построенное дерево, удаляет одно из поддеревьев и на его место копирует любое другое поддерево (возможно, другого размера), заменив все a_i из поддерева на $a_i \oplus k$. Для лучшего понимания ниже дана иллюстрация **одного** редактирования с $k = 4$.

Бараш хочет как можно скорее прочитать *шедевральный* стих друзьям. Помогите ему найти минимальное число редактирований или сообщите, что сделать стих *шедевральным* невозможно.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных.

В первой строке каждого набора даны два целых числа n и k ($1 \leq n \leq 20, 0 \leq k \leq 2^{31} - 1$).

Во второй строке каждого набора даны 2^n целых чисел $a_1, a_2, \dots, a_{2^n}$ ($0 \leq a_i \leq 2^{31} - 1$) — количества слогов.

Гарантируется, что сумма 2^n по всем наборам входных данных не превосходит 2^{20} .

Выходные данные

Для каждого набора входных данных выведите минимальное количество редактирований или -1 , если получить *шедевральный* стих невозможно.

входные данные
8
3 4
2 7 3 9 2 3 5 4
2 4
1 2 3 4
2 1
2 3 2 3
2 0
2 2 1 1
2 1
0 0 0 0
2 1
3 3 3 1
1 13
2 4
2 1
0 1 0 0
выходные данные
1
0
2
-1
2
1
0
2

Первый пример совпадает с картинкой из условия.

J. Не может быть

1 секунда, 256 мегабайт

Рядом с домиком Лосяша был обнаружен неведомый цветок инопланетного происхождения.

Лосяш очень им заинтересовался. Если существование цветка не Loading [MathJax]/extensions/TeX/mathchoice.js то он будет уничтожен.

Довольно быстро Лосяш смог определить параметр цветка: число n .

Проведя множество бессонных ночей за исследованиями, он смог вывести критерий того, что существование цветка поддается научному объяснению. Оно поддается научному объяснению тогда и только тогда, когда существует такое целое положительное число x , что среди чисел $x, x + 1, \dots, x + n - 1$ нет числа, взаимно простого со всеми остальными. Другими словами, должно существовать такое целое число x ($1 \leq x$), что не существует целого числа i ($0 \leq i \leq n - 1$), такого, что для любого j ($0 \leq j \leq n - 1, i \neq j$) $\gcd(x + i, x + j) = 1$.

Лосяшу известно, что число x может быть очень большим, поэтому он хочет найти остатки от деления x на все простые числа, не превосходящие n . Для публикации научной статьи этого будет достаточно.

Помогите Лосяшу найти x и определить судьбу цветка.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 2500$) — количество наборов входных данных.

В единственной строке каждого набора дано целое число n ($2 \leq n \leq 5000$) — параметр цветка.

Гарантируется, что сумма n по всем наборам входных данных не превосходит 5000.

Выходные данные

Для каждого набора входных данных выведите « -1 », если числа x не существует.

В противном случае выведите в первой строке количество простых чисел, не превосходящих n . Во второй строке выведите остатки от деления x на эти простые числа по порядку ($x \bmod 2, x \bmod 3, x \bmod 5, \ldots$). Если ответов несколько, вы можете вывести любой.

входные данные
2
2
20
выходные данные
-1
8
0 2 0 5 0 10 0 0

Во втором примере подходит, например, число $x = 7745540$. Его остатки от деления на первые восемь простых чисел — это остатки в ответе.

К. Личная жизнь

2 секунды, 256 мегабайт

Копатыч тайно готовится к вторжению марсиан.

Участок Копатыча представляет собой клетчатое поле из n строк и m столбцов. Копатыч заранее установил $2 \cdot k$ систем вентиляции в различных позициях $(x_1, y_1), (x_2, y_2), \ldots, (x_{2k}, y_{2k})$.

Для длительной обороны необходимо выкопать k подкопов. Каждый подкоп должен соединять какую-то пару систем вентиляции. Каждая система вентиляции должна быть частью ровно одного подкопа. Чтобы система подкопов была отказоустойчивой, никакие два подкопа не должны пересекаться. Другими словами, каждая клетка участка может принадлежать не более чем одному подкопу.

Помогите Копатычу подготовить подкопы для обороны.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных.

В первой строке каждого набора даны целые числа n, m, k ($1 \leq n, m \leq 500, 2 \leq \frac{n \cdot m}{k}$, $1 \leq k \leq \frac{n \cdot m}{2}$) — количество строк, столбцов и подкопов.

В следующих $2 \cdot k$ строках каждого набора даны пары чисел $(x_1, y_1), (x_2, y_2), \ldots, (x_{2k}, y_{2k})$ ($1 \leq x_i \leq n, 1 \leq y_i \leq m$) — позиции систем вентиляции.

Гарантируется, что сумма $n \cdot m$ по всем наборам входных данных не превосходит 250\,000.

Выходные данные

Для каждого набора входных данных выведите описания k подкопов.

Описание каждого подкопа должно содержать в первой строке число s_i — количество клеток участка в i -м подкопе.

Следующие s_i строк должны содержать по два целых числа x_i, y_i ($1 \leq x_i \leq n, 1 \leq y_i \leq m$) — описание клеток подкопа. Первая и последняя клетки должны содержать системы вентиляции. Соседние клетки должны быть соседними по стороне.

входные данные
3 3 3 2 1 1 1 3 3 1 3 3 3 5 3 3 3 1 1 1 3 1 4 2 5 2 1 4 4 1 4 1 1 4

выходные данные
3 1 1 1 2 1 3 3 3 1 3 2 3 3 2 1 1 2 1 2 1 3 1 4 4 3 3 3 4 3 5 2 5 7 4 1 3 1 2 1 1 1 1 2 1 3 1 4

В первом наборе входных данных мы можем соединить подкопом первую систему вентиляции со второй, подкопом с координатами (1, 1), (1, 2), (1, 3), а третью систему вентиляции с четвертой подкопом с координатами (3, 1), (3, 2), (3, 3) (см. картинку).